

Flutter 4: Juego Pong

Contenidos

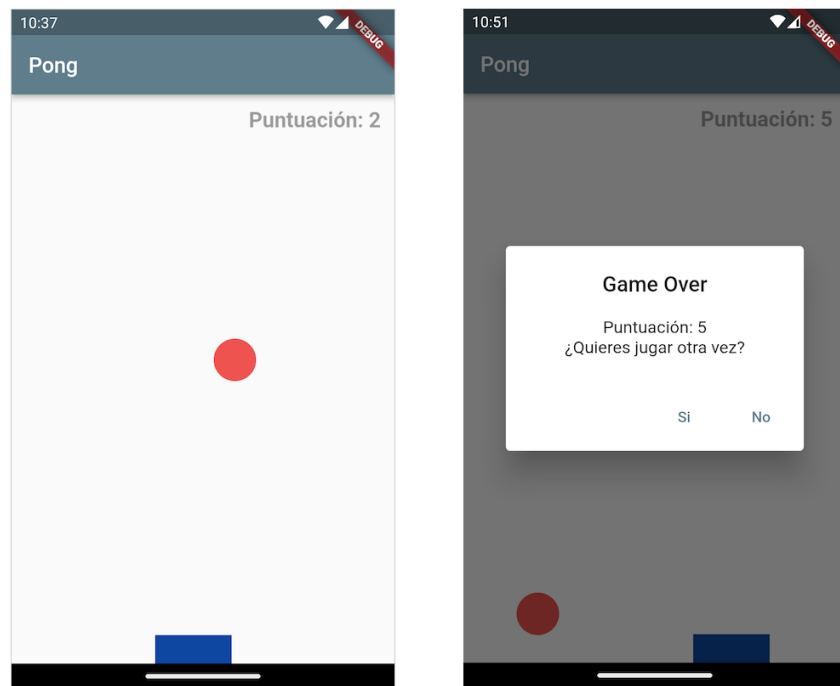
- Flutter 4: Juego Pong
 - ¿Qué voy a hacer en esta sesión?
 - 1 – Escribir una clase aplicación básica
 - 2 – Escribir la clase pelota
 - 3 – Escribir la clase raqueta
 - 4 – Escribir la rejilla de juego
 - 5 – Dibujar la pelota y la raqueta sobre la rejilla del juego
 - 6 – Averiguar el tamaño de la rejilla del juego
 - 7 – Usar animaciones para mover vistas por la pantalla
 - Teoría sobre las animaciones en Flutter
 - Una animación sencilla en Flutter
 - 8 – Añadiendo movimiento continuo a la pelota
 - 9 – Moviendo la raqueta
 - 10 – Coordinar la raqueta y la pelota
 - 11 – Toques finales: 1 puntuación
 - 12 – Toques finales: 2 Preguntar si comenzar nueva partida
 - Mini ejercicios

¿Qué voy a hacer en esta sesión?

En esta sesión vamos a construir una versión sencilla del famoso juego Pong, lanzado en 1972 por Atari y escrito por Nolan Bushnell.

Pong es un juego muy sencillo, pero su programación nos dará el conocimiento necesario para programar cualquier juego 2D en Flutter, usando el mecanismo potente y al mismo tiempo sencillo de utilizar, que incorpora Flutter para gestionar animaciones.

El aspecto de la aplicación terminada es este:



Aspecto de las pantallas principales del juego Pong.

Mientras la escribimos paso a paso aprenderemos:

- A utilizar el mecanismo de animaciones de Flutter para generar eventos a intervalos regulares de tiempo que pueden escucharse para controlar una animación en un juego.
- A averiguar el tamaño, expresado en dp, de una vista que se muestra en pantalla, teniendo en cuenta las proporciones del dispositivo físico y el tamaño que ya ocupan los ancestros de esa vista en el árbol de vistas.
- A mostrar opciones a un usuario mediante un cuadro de diálogo y a responder a su elección.
- A utilizar entre otros los widgets de Flutter: `LayoutBuilder`, `Stack`, `Animation`, `AnimationController`, `Tween`, `GestureDetector`, `AlertDialog` y `TextButton`

1 – Escribir una clase aplicación básica

Al igual que en otras sesiones de prácticas, empezaremos construyendo un nuevo proyecto de Flutter. Yo le he llamado `flutter_sesion_4_final`. Limpiaremos todo el código de la función `main` en el archivo `main.dart` y escribiremos una clase aplicación: `PongApp` en su propio archivo `pong_app.dart`.

Como siempre, esta clase es subclase de `StatelessWidget` y en su método `build` debe devolver una instancia de `MaterialApp`. En este caso, el `Scaffold` que organiza las distintas partes de la interfaz lo escribiremos directamente en la clase `PongApp`.

Después de hacer todo eso, el contenido de la clase `PongApp` es:

```
class PongApp extends StatelessWidget {
  const PongApp({Key? key}) : super(key: key);

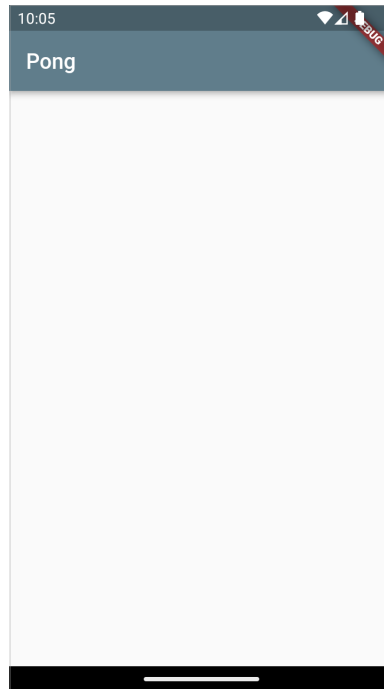
  @override
  Widget build(BuildContext context) {
    final titulo = 'Pong';

    return MaterialApp(
      title: titulo,
      theme: ThemeData(
        primarySwatch: Colors.blueGrey,
      ),
      home: Scaffold(
        appBar: AppBar(
          title: Text(titulo),
        ),
        body: Container(),
      ),
    );
  }
}
```

y en la función `main` escribiremos la llamada para ejecutar esa aplicación:

```
void main() {
  runApp(PongApp());
}
```

Si compilamos y ejecutamos la aplicación, el resultado debería ser similar a éste:



Aspecto de la aplicación Pong tras el paso 1

2 – Escribir la clase pelota

A continuación escribiremos una clase que contendrá una vista que representa a la pelota. Será una instancia de `StatelessWidget`. El código completo se escribe en su propio archivo `pelota.dart` y tiene el siguiente aspecto:

```
class Pelota extends StatelessWidget {  
  final double diametro;  
  Pelota({Key? key, this.diametro = 50});  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      width: diametro,  
      height: diametro,  
      decoration: BoxDecoration(  
        color: Colors.red[400],  
        shape: BoxShape.circle,  
      ),  
    );  
  }  
}
```

Veamos algunas consideraciones interesantes con respecto a este código:

- Puede parecer raro que se haya hecho `Pelota` subclase de `StatelessWidget`, una vista sin estado. La pelota deberá moverse por la pantalla y por tanto debería tener un estado que cambia.

Este razonamiento tiene lógica, pero no es correcto. La pelota no tiene información sobre el tamaño de la pantalla sobre la que se mueve, tampoco sobre los otros objetos con los que interactúa, por tanto, si se le diese la responsabilidad de moverse, a la fuerza se está acoplando con otros elementos del juego. No es una buena idea.

La pelota tiene la responsabilidad de dibujarse con el color, forma y tamaño adecuado, por eso se encapsula en su propia clase. Será la pantalla del juego quien tendrá la responsabilidad de saber dónde y cómo se dibuja la pelota.

Nota: Los elementos gráficos del juego (pelota y raqueta), se dibujarán como vistas móviles situadas sobre una pantalla.

- El diámetro de la pelota, se almacena como una propiedad de la clase `Pelota`. Se le ha dado un valor inicial de 50, pero puede cambiarse por el que se quiera dando valor al parámetro `diámetro` en el constructor.
- Usaremos la clase `Container` para representar la pelota. Esta clase es muy versátil ya que es una vista contenedora a la que se puede aplicar un tamaño y una decoración. Tiene varias propiedades interesantes de las que en esta aplicación hemos usado tres:
 - `width`: para indicar la anchura de la pelota. Se le asigna como valor el diámetro.
 - `height`: para indicar la altura de la pelota. Se le asigna como valor el diámetro.
 - `decoration`: donde se especifica la decoración que se usará para dibujar el fondo del contenedor. Si hubiera una vista hija, se dibujaría sobre ese fondo. En este caso no hay ninguna vista dentro del contenedor.

Como decoración se puede elegir cualquier subclase de la clase abstracta `Decoration`, en concreto se ha usado una vista de tipo `BoxDecoration` para la que se pueden especificar muchas propiedades que permiten dar un color, una forma para la caja, una imagen para el fondo, etc.

En este caso se ha elegido un color sólido mediante la propiedad `color` y una forma circular, mediante la propiedad `shape` a la que se pasa la constante `circle` del tipo enumerado `BoxShape`

- Para especificar el color de la pelota se ha usado la clase `Colors` que contiene la paleta de colores de **Material Design**. La mayoría de los colores tienen variantes que se especifican mediante un número entre corchetes. Ese número va del 100 al 900 con incrementos de 100 mas el valor 50. Permiten establecer matices sobre el color base. Los número mayores representan colores más intensos y los número menores representan colores mas pálidos.

Nota: En el caso de los colores con acento, por ejemplo `redAccent` el número de variantes es menor, solo 100, 200 400 y 700.

Con eso tenemos la pelota lista para ser usada. En breve la dibujaremos en la pantalla.

3 – Escribir la clase raqueta

De forma similar a como hemos escrito la clase pelota, escribiremos la clase `Raqueta` en su propio archivo `raqueta.dart`. Usaremos la vista `Container` para dibujarla y no es necesario gestionar un estado mutable ya que será el tablero del juego el que se encargará de gestionar la raqueta.

El contenido del archivo debería ser:

```
class Raqueta extends StatelessWidget {
  final double anchura;
  final double altura;

  Raqueta({Key? key, this.anchura = 100, this.altura = 25});

  @override
  Widget build(BuildContext context) {
    return Container(
      width: anchura,
      height: altura,
      decoration: BoxDecoration(
        color: Colors.blue[900],
      ),
    );
  }
}
```

Como es lógico, hay algunas diferencias con respecto al código de la clase `Pelota`:

- La raqueta es un rectángulo, por tanto se necesitan dos valores diferentes para la anchura y la altura. Se les ha dado un valor inicial de 100 y 25 respectivamente, pero no es relevante ya que se cambiará mas adelante. El tamaño de la raqueta se calculará en función del tamaño en dp del dispositivo real en el que se ejecute la aplicación.
- No se ha usado la propiedad `shape:` dentro de la clase `BoxDecoration` ya que el valor por defecto es `BoxShape.rectangle` que es justo el que se necesita.

Con todo esto, ya se tiene terminada y lista para usar la raqueta. En próximas secciones la dibujaremos en la pantalla.

4 – Escribir la rejilla de juego

La tercera clase que se necesita es la rejilla del juego sobre la que se moverán la pelota y la raqueta. En este caso es una subclase de `StatefulWidget` porque la rejilla necesita controlar dónde se encuentran la pelota y la raqueta y refrescar la interfaz cuando alguna de ellas se mueva. Ese es su estado mutable.

Añadiremos al proyecto un nuevo archivo: `rejilla_juego.dart` y en él escribiremos la clase `RejillaJuego` :

```
class RejillaJuego extends StatefulWidget {
  const RejillaJuego({Key? key}) : super(key: key);

  @override
  State<RejillaJuego> createState() => _RejillaJuegoState();
}

class _RejillaJuegoState extends State<RejillaJuego> {
  @override
  Widget build(BuildContext context) {
    return Container();
  }
}
```

El primer cambio que haremos a esta clase será cambiar la vista de tipo `Container` devuelta en el método `build` por una vista de tipo `LayoutBuilder` cuya vista hija será una vista de tipo `Stack` . Para ello sustituiremos la línea:

```
return Container();
```

por las líneas:

```
return LayoutBuilder(
  builder: (BuildContext context, BoxConstraints constraints) {
    return Stack();
  }
);
```

Veamos algunas consideraciones interesantes sobre esas dos clases:

- La vista `LayoutBuilder` es una vista contenedora (dentro se puede colocar otra vista) cuyo interés en este caso es que permite averiguar cuál es el espacio disponible con el que cuenta.

Para ello tiene en cuenta el espacio de la pantalla del dispositivo que ya han ocupado las vistas que son sus ancestros en el árbol de vistas, así como las restricciones que se aplican (por ejemplo una `SafeArea` que impide que los elementos del Sistema Operativo del dispositivo se dibujen superpuestos sobre las vistas de la aplicación)

Permite conocer el espacio disponible medido en dp. Más adelante usaremos esta característica para averiguar si la pelota ha chocado contra las "paredes" de la rejilla del juego, o para definir el tamaño de la raqueta de forma que sea proporcional al tamaño del dispositivo.

- La vista `Stack` es una vista contenedora (puede contener una lista de vistas) cuyo interés para programar el juego es que permite colocar a sus vistas hijas en posiciones que son relativas a sus bordes, es decir, a una determinada distancia medida en dp del borde superior, inferior, izquierdo o derecho.

Para situar una vista hija (por ejemplo la pelota) dentro de una vista `Stack` no hay que especificar una posición absoluta, sino una distancia relativa a alguno de sus bordes.

Para que la rejilla de juego sea visible, hay que incluirla en el árbol de vistas de la aplicación. Para ello, en la clase `PongApp` buscaremos la línea:

```
body: Container(),
```

y la sustituiremos por:

```
body: const SafeArea(  
  child: RejillaJuego(),  
)
```

Nota: Todavía no se puede ver nada en la pantalla porque aún no hemos dibujado ningún elemento del juego (pelota y/o raqueta) sobre la rejilla. Lo haremos en la siguiente sección.

5 – Dibujar la pelota y la raqueta sobre la rejilla del juego

Es el momento de añadir las vistas hijas a la clase `Stack` que se devuelve como contenido de la clase `LayoutBuilder` que representa a la rejilla del juego. Para ello se usa la vista `Positioned`.

En la clase `RejillaJuego` buscaremos la línea:

```
return Stack();
```

y las sustituiremos por las líneas:

```
return Stack(  
  children: <Widget>[  
    Positioned(  
      child: Pelota(),  
      top: 0,  
    ),  
    Positioned(  
      child: Raqueta(),  
      bottom: 0,  
    ),  
  ],  
)
```

Veamos algunas consideraciones importantes sobre este código:

- Se usa la propiedad `children:` de la vista `Stack`, a la que se pasa una lista de `Widget`, para añadir los elementos del juego (pelota y raqueta) a la rejilla del juego.

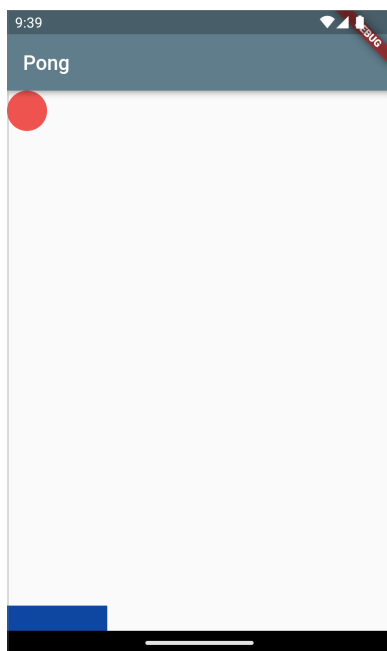
- Para poder determinar su posición dentro del `Stack` se usa una vista de clase `Positioned` que es una vista contenedora. La función de la vista `Positioned` es controlar dónde su vista hija se coloca dentro de una vista `Stack`. Tiene dos propiedades:
 - `child:` donde se añade la vista que se quiere *posicionar* en el `Stack`
 - `top:`, `bottom:`, `left:` o `right:` para especificar una distancia relativa al borde superior, inferior, izquierdo y derecho respectivamente.

Se pueden usar dos de estas propiedades, para por ejemplo, situar una vista hija a una determinada distancia del borde superior y del borde izquierdo.

Consejo: Te recomiendo que pruebes distintas combinaciones para ver en la práctica el efecto de estas propiedades.

En este caso se ha pintado la pelota a distancia `0` del borde superior y la raqueta a distancia `0` del inferior.

Si hacemos hot reload, veremos la pelota y la raqueta en nuestra pantalla.



Pelota y raqueta sobre la rejilla del juego.

Como vemos, la aplicación todavía es muy básica, pero ya empieza a tener el aspecto final que se busca.

6 – Averiguar el tamaño de la rejilla del juego

Por ahora, se ha dado un tamaño arbitrario fijo a la raqueta (100 x 25). El tamaño se mide en dp, lo que significa que en todos los dispositivos se va a ver igual de grande, sin embargo las proporciones de pantalla pueden variar entre dispositivos lo que hará que en unos quede más espacio sin cubrir por la raqueta que en otros. Esto hace que jugar en unos sea más difícil que en otros. No es la mejor opción.

Es mejor dar a la raqueta un tamaño proporcional de forma que en todos los dispositivos, la proporción de espacio cubierto por la raqueta, en relación al espacio sin cubrir sea la misma (aunque la raqueta se muestre con tamaño diferente)

Para ello se usa la vista `LayoutBuilder` y en concreto el parámetro `constraints` de clase `BoxConstraints` que se pasa a la propiedad `builder:`

Para usarlo comenzaremos añadiendo dos nuevas propiedades: `anchoRejilla` y `altoRejilla` a la clase `_RejillaJuegoState`

```
double anchoRejilla = 0.0;
double altoRejilla = 0.0;
```

que se inicializarán dentro del método `build`. Justo delante de:

```
return Stack(
```

escribiremos:

```
anchoRejilla = constraints.maxWidth;
altoRejilla = constraints.maxHeight;
```

Veamos algunas consideraciones sobre ese código:

- La vista `LayoutBuilder` tiene una propiedad `builder:` que recibe una función anónima con dos parámetros: `context` donde se guarda el contexto del árbol de vistas de la aplicación y `constraints` donde se almacenan las restricciones de tamaño de la vista.
- Entre las restricciones hay dos que son las que nos interesan aquí: `maxWidth` que almacena la anchura máxima de la vista en dp para el dispositivo en el que se está ejecutando la aplicación y `maxHeight` que almacena la altura máxima. En ambos casos se tiene en cuenta la parte ya ocupada por sus ancestros en el árbol de vistas.

En las propiedades `anchoRejilla` y `altoRejilla` se tiene la anchura y la altura en dp de la rejilla de juegos. Se pueden usar para dar tamaño proporcional a la raqueta y que en todos los dispositivos el área cubierta por la raqueta con respecto al área no cubierta sea igual. Para ello añadiremos otras dos propiedades:

```
double anchoRaqueta = 0.0;
double altoRaqueta = 0.0;
```

y las inicializaremos justo delante de:

```
return Stack(
```

de la siguiente forma:

```
anchoRaqueta = anchoRejilla / 5.0;
altoRaqueta = altoRejilla / 20.0;
```

A continuación, sustituiremos el constructor de la `Raqueta` que se usa actualmente:

```
child: Raqueta(),
```

por este otro que hace uso de esas propiedades recién calculadas:

```
child: Raqueta(anchura: anchoRaqueta, altura: altoRaqueta,),
```

Consejo: En este ejemplo, se ha optado porque la raqueta ocupe una quinta parte de la anchura del dispositivo y una veinteva parte de su altura. Te recomiendo que pruebes otras proporciones y ejecutes la aplicación en diferentes emuladores con diferentes proporciones de pantalla, para ver el resultado.

También añadiremos una propiedad llamada `diametroPelota` para tener un mejor control sobre si la pelota ha sido golpeada por la raqueta o si el usuario ha fallado.

Añadiremos la declaración:

```
late double diametroPelota;
```

en el método `initState` se inicializará de la forma:

```
diametroPelota = 40.0;
```

y se usará en el constructor de la pelota de la forma:

```
child: Pelota(diametro: diametroPelota),
```

7 – Usar animaciones para mover vistas por la pantalla

Llegados a este punto se tiene una clase para representar la rejilla de juego y dos clases para representar los elementos del juego (pelota y raqueta). Es el momento de darles movimiento, pero antes veamos cómo funcionan las animaciones en Flutter.

Teoría sobre las animaciones en Flutter

Flutter proporciona tres clases `Animation`, `AnimationController` y `Tween` que juntas permiten mover vistas por la pantalla de forma fluida. Veamos cómo funciona cada una y después escribiremos un ejemplo sencillo para ver su funcionamiento en acción.

- `Animation` es una clase abstracta (no se puede instanciar directamente) y genérica (define una familia de clases ya que se puede asociar a otros tipos). Uno de los tipos que más se pueden usar es `Animation<double>`. Tiene un valor y un estado. Su tarea es generar secuencialmente números interpolándolos entre dos valores, durante una cierta duración.

Es la base de cualquier animación en Flutter, pero solo genera los números que se usan para controlar la animación. **No está ligada a ninguna vista en la pantalla** ni sabe nada de ellas.

Nota: La clase `Animation` de Flutter es muy potente. La salida de un objeto `Animation` pueden ser valores interpolados en forma lineal, como una curva, o cualquier otra proyección que se necesite. También se pueden obtener los números en orden inverso, o cambiar su dirección en mitad del proceso.

`Animation` es una clase genérica, su tipo asociado puede ser distinto de `double` como `Animation<Color>` o `Animation<Size>` lo que simplifica enormemente ciertos tipos de animación.

- `AnimationController` es una subclase de `Animation<double>` que genera un nuevo valor cada vez que el hardware esta preparado para un nuevo refresco. De esta forma no se desperdician cálculos que no van a poder mostrarse en pantalla.

Por defecto, un `AnimationController` produce linealmente números desde 0.0 a 1.0 durante una duración dada. Por ejemplo la declaración:

```
late AnimationController controladorAnimacion;

controladorAnimacion = AnimationController(
  duration: const Duration(seconds: 4), vsync: this,);
```

construye una instancia de `AnimationController` que durante 4 segundos produce números entre `0.0` y `1.0`

Cuando se crea un `AnimationController`, hay que pasar al constructor un argumento `vsync:`. Es un mecanismo ingenioso que previene que se generen animaciones para vistas que no se están mostrando en la pantalla en este momento. De esta forma se impide que se consuman recursos innecesarios. Se puede usar la `StatefulWidget` donde se muestra la animación como valor para el parámetro `vsync:` añadiendo `with SingleTickerProviderStateMixin` a la definición de la clase.

`AnimationController` es una subclase de `Animation<double>`, por eso se puede usar donde quiera que se necesite una instancia de la clase `Animation`. Sin embargo, `AnimationController` tiene métodos adicionales para controlar la animación, como `forward()`, que inicia la generación de números.

La generación de números está vinculada al refresco de la pantalla, normalmente son generados 60 números por segundo. Después de que se genera cada número, cada objeto `Animation` notifica el cambio a sus observadores, que generalmente son las vistas que deben refrescarse para mostrar el resultado de la animación en pantalla.

Otros métodos de `AnimationController` son `stop()` que detiene la generación de números y `reverse()` que inicia la secuencia de números pero en orden inverso (desde `1.0` hasta `0.0`)

- `Tween` . Por defecto, el objeto `AnimationController` produce números dentro del rango `[0.0, 1.0]` . Si se necesita un rango diferente o un tipo de datos diferente, se puede usar `Tween` para transformar la salida de un objeto `AnimationController` para que interpole un rango o tipo de dato diferente.

La clase `Tween` no tiene estado, solo tiene las propiedades `begin` y `end` . El único trabajo de la clase `Tween` es definir una proyección entre un rango de entrada y un rango de salida. El rango de entrada es normalmente `0.0` a `1.0` .

La clase `Tween` se asocia a una instancia de `AnimationController` para que transforme el valor por defecto en uno transformado al rango de valores que interese. Para eso se usa su método `animate` . Por ejemplo:

```
late Animation<double> animacion;

animacion = Tween<double>(begin: 0, end: 200).animate(contraladorAnimacion);
```

Genera un objeto de la clase `Tween` y lo asocia a la instancia de `AnimationController` que declaramos en la variable `contraladorAnimacion` un poco más arriba.

Cuando se interroga al objeto `Tween` por su valor con la instrucción:

```
animacion.value;
```

No se obtendrán valores entre `0.0` y `1.0` sino entre `0` y `200` que es el rango que se ha definido en el objeto `Tween`

- Como sabemos, la clase `Animation` y por tanto también su subclase `AnimationController` solo genera los números que se usan para controlar una animación, pero no está ligada a ninguna vista en la pantalla, ni sabe nada de ellas. Para poder usar los números generados para mover una vista, esa vista debe registrarse como observadora de la instancia de `AnimationController` . Para ello se usa:

```
animacion?.addListener(() {
  setState(() {
    // - Comprobar el número actual en la animación con: animacion.value
    //   y usarlo para cambiar alguna propiedad que forme parte del estado
    //   mutable de la vista
  });
});
```

Una animación sencilla en Flutter

Una vez que conocemos las clase principales para hacer animaciones en Flutter y cómo funcionan vamos a usarlas en un ejemplo sencillo. Escribiremos una animación que desplace la pelota en diagonal durante tres segundos, desde la esquina superior izquierda de la rejilla del juego (posición 0,0) hacia abajo y la derecha (posición 200,200).

1. Comenzaremos añadiendo cuatro nuevas propiedades a la clase: `_RejillaJuegoState` :

```
late Animation<double> animacion;  
late AnimationController controladorAnimacion;  
  
late double xPelota;  
late double yPelota;
```

Veamos ese código con un poco de detalle:

- Se han declarado dos variables `animacion` y `controladorAnimacion` . La primera contendrá la instancia de la clase `Tween` que permitirá convertir el rango de valores `0.0` a `1.0` al rango que nos interese (en este ejemplo `0` a `200` . La segunda variable contendrá la instancia de `AnimationController` que usaremos para generar los números que controlarán la animación.
- También hemos declarado otras dos variables `xPelota` e `yPelota` que forman parte del estado mutable de `_RejillaJuegoState` y que contendrán la posición en x e y de la pelota respectivamente.

Cuando `controladorAnimacion` genere un nuevo número, se notificará a los obsesrvadores, uno de ellos será la rejilla de juego que modificará los valores de `xPelota` e `yPelota` dentro de una llamada a `setState()` para forzar a Flutter a que se refresque la pantalla.

2. Para que `xPelota` e `yPelota` se usen para controlar la posición de la pelota, hay que sustituir:

```
Positioned(  
  child: Pelota(),  
  top: 0,  
)
```

por:

```
Positioned(  
  child: Pelota(),  
  top: yPelota,  
  left: xPelota,  
)
```

3. A continuación redefiniremos el método `initState` de la siguiente forma:

```
@override
void initState() {
  xPelota = 0.0;
  yPelota = 0.0;
  controladorAnimacion = AnimationController(
    duration: const Duration(seconds: 3),
    vsync: this,
  );
  super.initState();
}
```

Muy importante: En este punto veremos que hay un error de compilación en la propiedad `vsync`. Se debe a que espera una instancia de la clase `TickerProvider` pero el valor que se pasa `this` es de tipo `State<RejillaJuego>`

Se arregla sustituyendo la declaración de la clase:

```
class _RejillaJuegoState extends State<RejillaJuego> {
```

por:

```
class _RejillaJuegoState extends State<RejillaJuego>
  with SingleTickerProviderStateMixin {
```

La palabra reservada `with` se usa para hacer que una clase pueda usar los métodos de otra sin extenderla (relación de herencia). A este mecanismo se le denomina **Mixin** y su estudio se sale del ámbito de este guión de prácticas.

La clase `SingleTickerProviderStateMixin` proporciona a nuestra clase `_RejillaJuegoState` los métodos de un `Ticker` que es una clase que envía señales a intervalos regulares (60 veces por segundo).

Veamos este código con un poco de detalle:

- Las variables `xPelota` e `yPelota` se han inicializado con el valor `0.0` para que la animación comience justo en la esquina superior izquierda de la rejilla del juego.
- Se ha inicializado `controladorAnimacion` con una instancia de `AnimationController` con duración 3 segundos.

4. Lo siguiente es definir un objeto de clase `Tween` para transformar el rango de valores proporcionado por el `AnimationController` a las coordenadas que queramos que se mueva la pelota por la pantalla. Justo antes de:

```
super.initState();
```

escribiremos:

```
animacion = Tween<double>(begin: 0, end: 200).animate(controladorAnimacion);
```

- Como vemos, se asocia la instancia de la clase `Tween` con la instancia de la clase `AnimationController` que se tiene en `controladorAnimacion`. De esta forma la proyección se aplicará a los valores que produzca `controladorAnimacion`

5. A continuación registraremos a la clase `_RejillaJuegoState` para que observe los cambios del objeto `AnimationController`, de forma que se pueda refrescar la pantalla con los nuevos valores. Justo antes de:

```
super.initState();
```

escribiremos:

```
animacion.addListener(() {  
  setState(() {  
    xPelota=animacion.value;  
    yPelota=animacion.value;  
  });  
});
```

- Como vemos, al registrar la clase `_RejillaJuegoState` como observadora de la instancia de `Tween` que se tiene en `animacion`, se pasa una función anónima. Es el callback que se ejecutará cuando haya un nuevo número generado por la animación.

En este caso se debe refrescar la pantalla, por eso se llama al método `setState` para forzar a Flutter a que redibuje toda la interfaz, y dentro se cambia el valor del estado mutable guardado en `xPelota` e `yPelota`.

En este ejemplo sencillo, solo se les da el valor de `animacion.value`

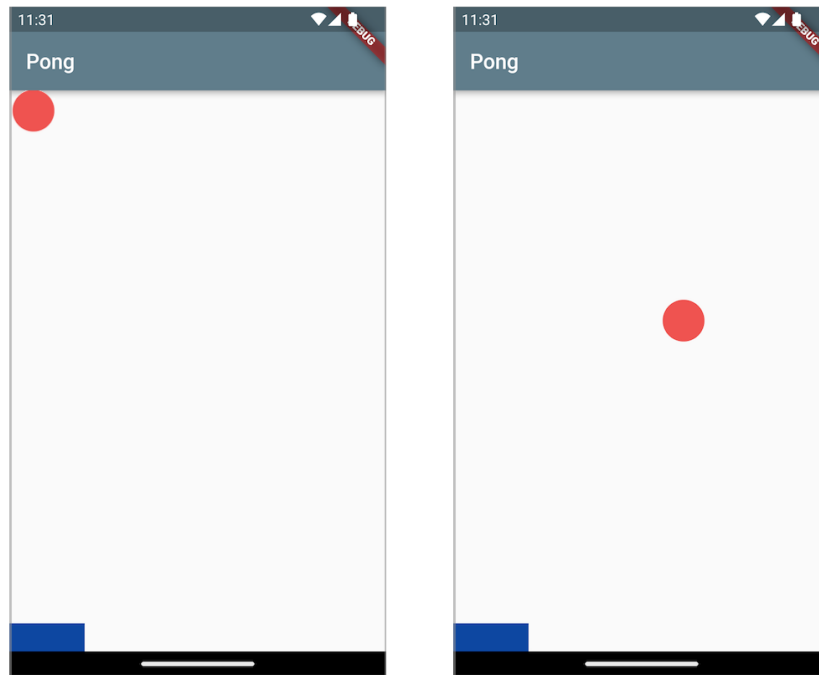
6. Para terminar pedimos a la animación que se ejecute. Justo delante de:

```
super.initState();
```

escribiremos:

```
controladorAnimacion.forward();
```


Si hacemos hot restart, veremos que la pelota se mueve durante tres segundos, dentro de la rejilla del juego, desde la posición `(0,0)` a la posición `(200,200)`



Primera animación en Flutter. La pelota se mueven en diagonal, desde la esquina superior izquierda, durante 3 segundos.

Consejo: Para entender mejor el efecto de estas clases prueba a cambiar los valores de las propiedades `begin` y `end` de la clase `Tween` y la duración de la animación. Observa el resultado. ¿Qué efecto tienen?

Recuerda usar hot restart después de cada cambio. En este caso hot reload no funciona.

8 - Añadiendo movimiento continuo a la pelota

La animación de ejemplo de la sección anterior, es vistosa, pero no sirve para el propósito del juego.

- En primer lugar, no debería parar a menos que se haya terminado la partida, por tanto, su duración no debería estar definida por un tiempo predeterminado, sino que debe estar en función de cómo se desarrolle el juego.
- En segundo lugar, no debería moverse siempre en diagonal, hacia abajo y a la derecha, sino que cuando golpee alguna pared de la rejilla de juegos, debería rebotar para moverse en dirección simétrica.

Como vemos, la posición de la pelota depende de la lógica del juego que indica que debe rebotar en las paredes, así que solo se usará la clase `AnimationController` para marcar el tiempo de la animación y no porque se vayan a usar los valores que producen para posicionar la pelota.

1. Comenzaremos declarando un tipo enumerado `Direccion`, con cuatro constantes que indican las cuatro direcciones principales. Para ello, en `rejilla_juego.dart` añadiremos:

```
enum Direccion {  
  arriba, abajo, izquierda, derecha,  
}
```

2. Lo siguiente es añadir a `_RejillaJuegoState` dos nuevas propiedades de tipo `Direccion`. En el método `initState` añadiremos:

```
direccionVertical = Direccion.abajo;  
direccionHorizontal = Direccion.derecha;
```

3. Dado que en este juego, la posición de la pelota no se toma directamente de los números que genera el `AnimationController`, ya no será necesario un `Tween` para convertir los números a un rango adecuado para mostrar la pelota en la pantalla. Por eso borraremos la declaración:

```
late Animation<double> animacion;
```

y la inicialización:

```
animacion = Tween<double>(begin: 0, end: 200).animate(contraladorAnimacion);
```

y sustituiremos las líneas:

```
animacion.addListener(() {  
  setState(() {  
    xPelota=animacion.value;  
    yPelota=animacion.value;  
  });  
});
```

por las líneas:

```
contraladorAnimacion.addListener(() {  
  setState(() {  
    (direccionHorizontal == Direccion.derecha) ? xPelota += incremento: xPelota -=  
    incremento;  
    (direccionVertical == Direccion.abajo) ? yPelota += incremento: yPelota -=  
    incremento;  
  });  
  comprobarBordes();  
});
```

Veamos con un poco de detalle esto último:

- Ya no se coloca como posición de la pelota el valor de la animación transformado por la clase `Tween`: `xPelota=animacion.value;`. En su lugar se le suma una cantidad fija que puede ser negativa o positiva en función de la dirección de desplazamiento:

```
(direccionHorizontal == Direccion.derecha) ? xPelota += incremento: xPelota -= incremento;
```

La cantidad fija que se suma a la posición de la pelota, se define en una propiedad de la clase `_RejillaJuegoState` llamada `incremento`. Para ello se añade la `línea`:

```
late double incremento;
```

y en el método `initState`:

```
incremento = 5.0;
```

- Después de actualizar la posición de la pelota, hay que comprobar si se ha chocado con alguno de los bordes. Para ello se usa el método `comprobarBordes()` que se describe en el siguiente punto.

La llamada a `comprobarBordes()` no se incluye dentro de `setState` porque no se debe refrescar la pantalla como resultado de este método.

4. A continuación añadiremos un método para comprobar si la pelota ha chocado contra alguna de las paredes. Si choca contra una pared lateral, hay que invertir la dirección horizontal y si choca contra la pared superior o inferior, hay que invertir la dirección vertical.

En la clase `_RejillaJuegoState` añadiremos el siguiente método:

```
void comprobarBordes() {  
    double bordeDerecho = anchoRejilla - diametroPelota;  
    double bordeInferior = altoRejilla - diametroPelota - altoRaqueta;  
  
    if (xPelota <= 0 && direccionHorizontal == Direccion.izquierda) {  
        direccionHorizontal = Direccion.derecha;  
    } else if (xPelota >= bordeDerecho && direccionHorizontal == Direccion.derecha) {  
        direccionHorizontal = Direccion.izquierda;  
    }  
  
    if (yPelota <= 0 && direccionVertical == Direccion.arriba) {  
        direccionVertical = Direccion.abajo;  
    } else if (yPelota >= bordeInferior && direccionVertical == Direccion.abajo) {  
        direccionVertical = Direccion.arriba;  
    }  
}
```

Veamos algunas consideraciones sobre este método:

- La rejilla del juego es de clase `Stack` que está contenida dentro de una clase `LayoutBuilder`. Esto le permite saber el tamaño en dp de la rejilla del juego. Para ello se usan las variables `anchoRejilla` y `altoRejilla` que se inicializaron con los valores:

```
anchoRejilla = constraints.maxWidth;  
altoRejilla = constraints.maxHeight;
```

- La posición horizontal de la pelota se mide desde el borde izquierdo. Por eso, cuando se comprueba si se ha golpeado el borde derecho, hay que descontar la anchura de la pelota. Para simplificar ese proceso, se usa la variable `bordeDerecho` inicializada de la forma:

```
double bordeDerecho = anchoRejilla - diametroPelota;
```

- La posición vertical de la pelota se mide con respecto al borde superior. Por eso, cuando se comprueba si se ha golpeado el borde inferior, hay que descontar la anchura de la pelota y de la raqueta. Para simplificar ese proceso, se usa la variable `bordeInferior` inicializada de la forma:

```
double bordeInferior = altoRejilla - diametroPelota - altoRaqueta;
```

- Como puede verse, se comprueba siempre si se ha golpeado un borde lateral y también si se ha golpeado un borde superior o inferior. Hay que comprobar los dos, ya que la colisión podría darse en una de las esquinas.
 - Sin embargo, si se comprueba que se ha golpeado uno de los lados, es seguro que el lado de enfrente no se va a haber golpeado. Por ejemplo, si se comprueba que se ha golpeado el lado izquierdo de la rejilla de juego, es seguro que no se va a haber golpeado también el lado derecho. Por eso se usa una comparación con `else if`
 - En el caso de colisión, se invierte la dirección del movimiento.
5. Lo siguiente es garantizar que la animación va a durar el tiempo suficiente para dar lugar al usuario a fallar. Lo fijaremos en 10.000 minutos, que es una cantidad de tiempo enorme, mas que suficiente para una partida. Para ello sustituiremos las líneas:

```
controladorAnimacion = AnimationController(  
    duration: const Duration(seconds: 3),  
    vsync: this,  
);
```

por las líneas:

```
controladorAnimacion = AnimationController(  
  duration: const Duration(minutes: 10000),  
  vsync: this,  
);
```

6. Por último, es muy importante redeclarar el método `dispose` de la clase `_RejillaJuegoState` para eliminar la instancia de `AnimationController`. Para ello se añade el método:

```
@override  
void dispose() {  
  controladorAnimacion.dispose();  
  super.dispose();  
}
```

Si se hace hot restart, veremos que ahora la pelota comienza a moverse y cuando llega hasta uno de los bordes de la rejilla del juego, rebota de forma simétrica. Si se dejase 10.000 minutos, veríamos como el juego termina cuando se cumple ese tiempo.

9 - Moviendo la raqueta

Ya tenemos una pelota móvil cuyo movimiento lo controla un objeto de la clase `AnimationController`. El movimiento es infinito, no termina nunca. Ahora debemos controlar también el movimiento de la raqueta.

La raqueta no se mueve de la misma forma que la pelota. La pelota es movida por una animación automáticamente, mientras que la raqueta es movida por el usuario deslizando sobre la pantalla.

Para controlar esto, se usa la clase `GestureDetector`, que como indica su nombre, es un widget que detecta los gestos que realiza el usuario sobre la vista. Se puede responder a muchos gestos de usuario. Los más comunes son `onTap`, `onDoubleTap` y `onLongPress`. Para responder a ellos se pasa una función anónima con el código que responde a ese gesto concreto.

En este juego, la raqueta solo puede desplazarse horizontalmente, por eso usaremos la propiedad `onHorizontalDragUpdate` de la clase `GestureDetector` que detecta e informa cada vez que se produce un desplazamiento horizontal sobre la vista. Para ello, en la clase `_RejillaJuegoState` buscaremos las siguientes líneas de código:

```
Positioned(  
  child: Raqueta(anchura: anchoRaqueta, altura: altoRaqueta,),  
  bottom: 0,  
),
```

y las reemplazaremos por las líneas:

```
Positioned(  
  bottom: 0,  
  left: xRaqueta,  
  child: GestureDetector(  
    onHorizontalDragUpdate: (DragUpdateDetails detalleDeslizar) {  
      moverRaqueta(detalleDeslizar);  
    },  
    child: Raqueta(anchura: anchoRaqueta, altura: altoRaqueta,)  
  ),  
)
```

También hay que añadir un nuevo método propio a la clase `_RejillaJuegoState` al que llamaremos `moverRaqueta` :

```
void moverRaqueta(DragUpdateDetails detalleDeslizar) {  
  setState(() {  
    xRaqueta += detalleDeslizar.delta.dx;  
    if (xRaqueta <= 0) {  
      xRaqueta = 0.0;  
    } else if (xRaqueta >= anchoRejilla - anchoRaqueta) {  
      xRaqueta = anchoRejilla - anchoRaqueta;  
    }  
  });  
}
```

Veamos este código con un poco mas de detalle:

- Se ha añadido una propiedad `xRaqueta` que contendrá la posición horizontal de la raqueta con respecto al lado izquierdo de la rejilla del juego. Se usa como valor de la propiedad `left:` de la vista `Positioned`.

Para poder usarlo, hay que añadir a `_RejillaJuegoState` esa propiedad:

```
late double xRaqueta;
```

e inicializarla en `initState` mediante la línea:

```
xRaqueta = 0.0;
```

- La propiedad `onHorizontalDragUpdate:` de la clase `GestureDetector` espera una función anónima con un parámetro de tipo `DragUpdateDetails` que contiene los detalles del gesto de deslizamiento que el usuario ha realizado sobre la vista.

A partir de esos datos se mueve la raqueta. Para ello se usa el método propio `moverRaqueta` que se describe a continuación.

- El método `moverRaqueta` usa los datos del gesto de deslizar que se le pasa como parámetro para actualizar el valor de `xRaqueta`. Para ello usa la propiedad `delta.dx` del parámetro de tipo `DragUpdateDetails` que se le pasa. En esa propiedad se tiene la distancia del desplazamiento en dp. Esa cantidad es negativa si el desplazamiento es a la izquierda y positiva si es a la derecha. Por eso solo hay que sumarlo:

```
xRaqueta += detalleDeslizar.delta.dx;
```

A continuación hay que comprobar que la raqueta no se ha movido fuera de los bordes de la rejilla de juego y en caso de que sea así, corregir la posición.

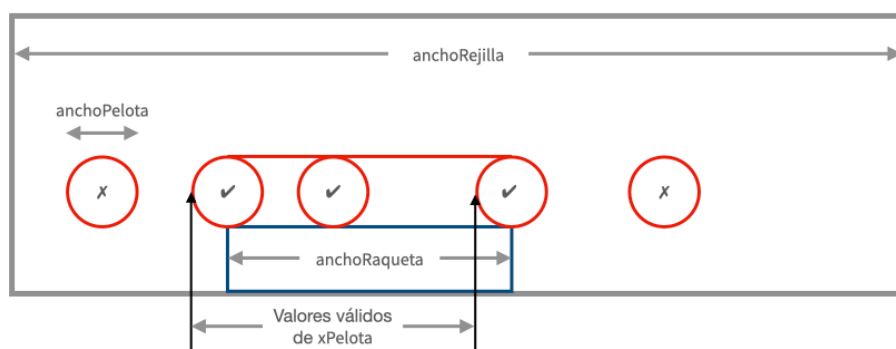
Todo ello se hace dentro de una llamada a `setState` porque se actualiza el valor de la propiedad `xRaqueta` que forma parte del estado mutable de la vista `RejillaJuego` y hay que forzar a Flutter a refrescar la interfaz.

Si hacemos hot restart, veremos que además de una pelota moviéndose de forma autónoma, en la rejilla de juego hay una raqueta que puede ser movida deslizando el dedo sobre ella. El juego va tomando forma, pero aún no hay coordinación entre el movimiento de la raqueta y el de la pelota.

10 - Coordinar la raqueta y la pelota

Tal y como está el juego en este momento, si la pelota choca con el borde inferior rebota hacia arriba en todos los casos. No es correcto. Lo correcto es comprobar si en el punto de impacto está la raqueta, en cuyo caso se produce el rebote o no está la raqueta, en cuyo caso se pierde la partida.

En esta versión simple del juego, consideraremos que la pelota ha rebotado contra la raqueta si está en alguna de las situaciones que se muestran en la siguiente imagen:



Valores válidos de `xPelota`. Si está fuera de ese rango (equivale a golpear fuera de la raqueta) se considera que se ha terminado la partida.

La orden para comprobar si se está dentro del rango es:

```
if (xPelota >= (xRaqueta - mitadPelota) &&
    xPelota <= (xRaqueta + anchoRaqueta - mitadPelota)) {
}
```

donde `mitadPelota` se calcula como:

```
double mitadPelota = diametroPelota / 2.0;
```

Esa comprobación debe hacerse dentro del método `comprobarBordes` cuando la pelota está en el borde inferior y moviéndose hacia abajo. Buscaremos las líneas actuales:

```
} else if (yPelota >= bordeInferior && direccionVertical == Direccion.abajo) {  
    direccionVertical = Direccion.arriba;  
}
```

donde se producía un rebote de forma incondicional y las sustituiremos por:

```
} else if (yPelota >= bordeInferior && direccionVertical == Direccion.abajo) {  
    if (xPelota >= (xRaqueta - mitadPelota) &&  
        xPelota <= (xRaqueta + anchoRaqueta + mitadPelota)) {  
        direccionVertical = Direccion.arriba;  
    } else {  
        controladorAnimacion.stop();  
        dispose();  
    }  
}
```

donde se comprueba si la pelota ha golpeado sobre la raqueta. Si es así, se produce el rebote, pero si no, se termina la partida.

Como podemos ver, para parar la partida se ejecuta:

```
controladorAnimacion.stop();  
dispose();
```

La construcción de las vistas de la aplicación y la animación se ejecutan en hebras distintas. Podría darse el caso de que la animación notifique a las vistas que hay un nuevo fotograma que refrescar, pero todavía la vista `LayoutBuilder` aún no tenga disponible la información de su anchura y altura. En ese caso `anchoRejilla` que se inicializa con el valor `constraints.maxWidth` y/o `altoRejilla` que se inicializa con el valor `constraints.maxHeight` podrían tener valor `0.0`. En ese caso, la comprobación de los bordes podría hacer que el juego pensara que ha terminado la partida porque la pelota está en el borde inferior y no se ha golpeado la raqueta, con lo que se pararía la animación y el juego ni siquiera comenzaría.

Para impedir que eso ocurra, se puede añadir la comprobación:

```
if (anchoRejilla == 0.0 || altoRejilla == 0.0) { return; }
```

justo al comienzo del método `comprobarBordes`

Si hacemos hot restart veremos que la pelota se mueve de forma automática mediante una animación, la raqueta se mueve mediante gestos de deslizamiento horizontal y si al llegar al borde inferior, la pelota no golpea la raqueta, el juego se para.

11 - Toques finales: 1 puntuación

Un juego no es muy interesante si no produce un resultado comparable que pueda medirse. Es la puntuación. Se pueden establecer muy diversos criterios de puntuación. En esta versión simple del juego, hemos optado por sumar un punto cada vez que el usuario golpea la pelota con la raqueta.

Necesitaremos una variable que contenga la puntuación actual. Es parte del estado mutable de `RejillaJuego` ya que cada vez que se cambie se debe mostrar refrescada en la pantalla.

También será necesaria una vista donde colocar la puntuación. Una forma sencilla es usar la propia rejilla de juegos para mostrarla.

1. Comenzaremos añadiendo una nueva propiedad a la clase `_RejillaJuegoState` llamada `puntuacion`:

```
int puntuacion = 0;
```

2. A continuación añadiremos una nueva vista a la rejilla del juego donde mostrar la puntuación. Para ello añadiremos lo siguiente a la lista de `Widget` hijos de la vista `Stack`:

```
Positioned(  
  top: 12,  
  right: 12,  
  child: Text(  
    'Puntuación: $puntuacion',  
    style: const TextStyle(  
      fontSize: 20,  
      fontWeight: FontWeight.bold,  
      color: Colors.black38,  
    ),  
  ),  
)
```

3. Por último en el método `comprobarBordes` junto debajo de:

```
direccionVertical = Direccion.arriba;
```

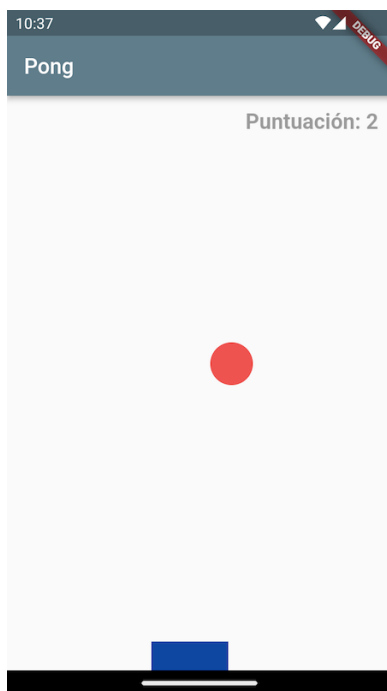
añadiremos:

```
setState(() {  
  puntuacion++;  
});
```

eso hará que se sume `1` a la puntuación actual y que se notifique a Flutter que hay que refrescar la interfaz para mostrar esa nueva puntuación.

Si hacemos hot restart veremos un nuevo texto con la puntuación en la parte superior de la rejilla del juego que va aumentando a medida que se van dando golpes a la pelota con la raqueta.

Si se ejecuta hot restart se puede ver el juego con la puntuación superpuesta sobre la rejilla del juego.



Aspecto del juego cuando se superpone la puntuación obtenida sobre la rejilla del juego.

12 - Toques finales: 2 Preguntar si comenzar nueva partida

Lo normal cuando una partida termina es que el juego informe de la puntuación lograda y pregunte si se quiere comenzar una nueva.

Eso puede hacerse de forma muy sencilla en Flutter con la clase `AlertDialog` y el método que lo muestra `showDialog`

Para utilizarlo, añadiremos un nuevo método llamado `preguntarRepetirPartida` a la clase `_RejillaJuegoState` con el siguiente contenido:

```
void preguntarRepetirPartida(BuildContext context) {
  showDialog(
    context: context,
    builder: (BuildContext context) {
      return AlertDialog(
        title: const Text(
          'Game Over',
          textAlign: TextAlign.center,
        ),
        content: Text(
          'Puntuación: $puntuacion\n¿Quieres jugar otra vez?',
          textAlign: TextAlign.center,
        ),
      ),
    ),
  );
}
```

```

actions: <Widget>[
  TextButton(
    child: const Text('Si'),
    onPressed: () {
      setState(() {
        xPelota = 0.0;
        yPelota = 0.0;
        puntuacion = 0;
      });
      Navigator.of(context).pop();
      controladorAnimacion?.repeat();
    },
  ),
  TextButton(
    child: const Text('No'),
    onPressed: () {
      Navigator.of(context).pop();
      dispose();
    },
  ),
],
);
}
);
}

```

y en el método `comprobarBordes` sustituiremos las líneas:

```

} else {
  controladorAnimacion.stop();
  dispose();
}

```

por las líneas:

```

} else {
  controladorAnimacion.stop();
  preguntarRepetirPartida(context);
}

```

Cuidado: Fíjate que la llamada a `dispose()` se ha eliminado del método `comprobarBordes` y se ha trasladado al método `preguntarRepetirPartida`

Veamos cómo funciona un cuadro de alerta en Flutter:

- Un cuadro de alerta en Flutter es una instancia de la clase `AlertDialog` . Para mostrarlo se usa el método: `showDialog`

`showDialog` es una función que muestra un diálogo de Material Design por encima del contenido actual de la aplicación, usando las animaciones de entrada y salida definidas en Material Design.

Tiene como parámetro un `BuildContext` que se utiliza para buscar el `Navigator` y el `theme` del árbol de vistas de la aplicación y con ellos construir el diálogo.

- La clase `AlertDialog` tiene varias propiedades aunque por lo general se usan tres:
 - `title`: donde se pasa una vista de tipo `Text` con el título que se quiera para el diálogo de alerta. Al texto se le puede aplicar un estilo como a cualquier otro texto.
 - `content`: donde se pasa otra vista de tipo `Text` que se usa para dar indicaciones más precisas al usuario sobre las acciones que puede tomar.
 - `actions`: que recibe una lista de `Widget`. Hay tantas como opciones se presenten al usuario. Por lo general son dos botones, los correspondientes a la acción de aceptar y cancelar.

Por lo general se usan vistas de clase `TextButton` para codificar estas acciones. Estas vistas tienen dos propiedades principales:

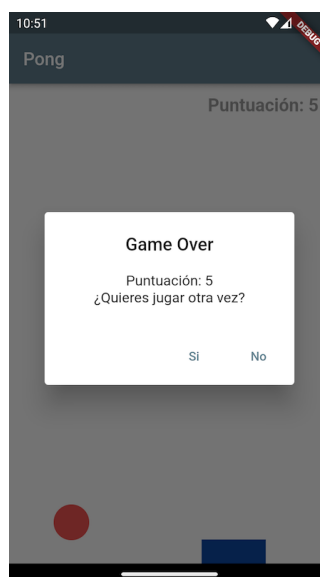
- `child`: donde se especifica el texto asociado al botón.
- `onPressed`: donde se especifica la función anónima que se debe ejecutar cuando el usuario pulse esa opción.

Muy importante: Siempre debe incluir una instrucción para cerrar el cuadro de diálogo y que no se siga viendo en la pantalla. Esto se consigue mediante:

```
Navigator.of(context).pop();
```

En esta versión sencilla del juego, si el usuario decide jugar otra partida se llama al método `repeat()` del `AnimationController` después de haber vuelto a situar la pelota en la esquina superior izquierda y haber puesto a cero la puntuación. Si el usuario decide no jugar, simplemente se cierra el cuadro de diálogo.

Con esto tendremos la aplicación terminada. Si hacemos hot restart, cuando tras algunos golpes se falle al golpear la pelota se mostrará un cuadro de diálogo similar a este:



Cuadro de diálogo que se muestra al terminar una partida.

Mini ejercicios

1. ¿Qué palabra reservada se usa para usar el mecanismo de Mixin en una clase?
2. ¿Cuál es la diferencia entre las clases de Flutter `Animation` y `AnimationController` ? ¿Y entre `AnimationController` y `Tween` ?
3. ¿En qué situaciones se usa la clase `Tween` ? ¿Por qué no se usa en el juego Pong?
4. ➡ Prueba a cambiar el valor de la variable `incremento` . Prueba con valores más grandes y más pequeños. ¿Qué efecto tiene en el juego? ¿A qué magnitud del juego equivale?
5. ➡ Haz una versión del juego mas interesante de jugar, en la que la velocidad de la pelota aumente a medida que se va golpeando con la raqueta.
6. ➡ Haz una versión del juego mas interesante de jugar, en la que el tamaño de la raqueta se reduzca a medida que se va golpeando la pelota con la raqueta. Comienza por ejemplo con una raqueta que ocupe el 50% de la anchura de la rejilla de juego y que se vaya reduciendo progresivamente.